

EFREI Paris – Mastercamp SR

Encadrant : Souheib Yousfi

TP3 : Hachage, Encodage et Chiffrement

Compte Rendu

Mazien Adhar

8 juin 2026

1. Introduction

Ce TP porte sur trois grandes notions de la sécurité informatique : le hachage, l'encodage et le chiffrement. L'idée c'est de les manipuler directement, à la fois en Python et en ligne de commande Linux, et ensuite de voir comment on peut chiffrer des emails avec Thunderbird et GPG. Tout a été fait sur une VM Ubuntu 26.04 sous VMware.

2. Fonctions de Hachage

2.1 Développement Python – script de hachage

On commence par exécuter le script `01_hachage_base.py` qui calcule le hash d'un mot de passe avec différents algorithmes. Ce qui est intéressant à observer ici, c'est que le même texte ne donne pas du tout la même empreinte selon l'algorithme utilisé. Et si on change juste une lettre ou un espace, le résultat change complètement – c'est ce qu'on appelle la propriété d'avalanche.

```
python3 01_hachage_base.py
```

```

maziem@ubuntu-VM: ~/TP3
maziem@ubuntu-VM:~/Bureau$ cp ~/Documents/pythonTP3/* ~/TP3/
bash: cp~/Documents/pythonTP3/*: Aucun fichier ou dossier de ce nom
maziem@ubuntu-VM:~/Bureau$ cp ~/Documents/pythonTP3/* ~/TP3/
maziem@ubuntu-VM:~/Bureau$ cd ~/TP3
maziem@ubuntu-VM:~/TP3$ ls
01_hachage_base.py          'attaque par dictionnaire.py'
02_encodage.py              bruteforce.py
03_chiffrement_symetrique_aes.py bruteforce_dictionnaire.py
04_chiffrement_asymetrique_rsa.py keylogger.py
05_hachage_salt_bcrypt.py
maziem@ubuntu-VM:~/TP3$ python3 01_hachage_base.py

SA4 - Script 01 : Fonctions de Hachage

=====
Texte : 'monmotdepasse'
=====
MD5      : 193f5287eff7d1d5e0a6e66ef769cb49
SHA-1    : dd629d39c4576731a2bef03c72ff89d6fc2a99a
SHA-256  : a7af71ad2b0ce07c36781ab7c8a6d36bd703824c22647f85d6de62063b219bc6
SHA-512  : 3d81b2c154b9be5d4f62f79a5156516d5a18ce13470c2181513feecdaee69956cdea30121e90d0e0cbd88541e0f73a
e3938f246c4b10900cb757b012a64b8985

=====
Texte : 'efrei2025'
=====
MD5      : 348c0a536163473e628ed95612dd164a
SHA-1    : 3ff36cd8df4638ded17a52ab96230daf931fead5
SHA-256  : b338a212bf3b376feb68f378b315951a2fbf18b0122a9394c9b8cca29d00e479
SHA-512  : 1426d5c10eb711bffa1dfed8dccc70a9bedae9107bc6102b35ffca9ce8de9e4cfe1faf75cf002984e3b309b2afb16
452fcf94497e57173de3ba49e37d9cbe76

=====
Texte : 'P@ssw0rd!'
=====
MD5      : 8a24367a1f46c141048752f2d5b14b
SHA-1    : 076d3e6c4b9f654b5b220b9045b7458ab6b4cbc6
SHA-256  : 0e44ce7308af2b3de5232e4616403ce7d49ba2aec83f79c196409556422a4927
SHA-512  : 9a585872fc4a94ba2fe0f7e9625bcfccc1050dab20b56df35a1c8915a3d8325616c0172284c2f4fe9471667c9c4f1f
bdc0d371684cae74cdf9d39d82383a383

--- DÉMONSTRATION : Sensibilité aux modifications ---
'hello' → SHA256 : 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
'Hello' → SHA256 : 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
'hello ' → SHA256 : 5e3235a8346e5a4585f8c58562f5052b8fe26a3bb122e1e96c76784964dfc461

Observation : même une majuscule ou un espace change totalement le hash !

--- LONGUEUR DES EMPREINTES ---
MD5      : 128 bits (32 caractères hex)
SHA1     : 160 bits (40 caractères hex)
SHA256   : 256 bits (64 caractères hex)
SHA512   : 512 bits (128 caractères hex)
maziem@ubuntu-VM:~/TP3$

```

Figure 1: Résultat du script – comparaison MD5, SHA-1, SHA-256 et SHA-512

2.2 Utilisation sous Linux

On utilise ensuite les commandes Linux pour faire la même chose directement dans le terminal. Le `|` (pipe) sert à envoyer la sortie de `echo` directement à la commande de hachage. Le `-n` est important : sans lui, `echo` ajoute un retour à la ligne à la fin du texte, ce qui changerait le hash.

```

echo -n "monmotdepasse" | md5sum
echo -n "monmotdepasse" | sha1sum
echo -n "monmotdepasse" | sha256sum

```

```

maziengubuntu-VM:~/TP3$ echo -n "monmotdepasse" | md5sum
193f5287eff7d1d5e0a6e66ef769cb49 -
maziengubuntu-VM:~/TP3$ echo -n "monmotdepasse" | sha1sum
dd629d39c4576731a2bef003c72ff89d6fc2a99a -
maziengubuntu-VM:~/TP3$ echo -n "monmotdepasse" | sha256sum
a7af71ad2b0ce07c36781ab7c8a6d36bd703824c22647f85d6de62063b219bc6 -
maziengubuntu-VM:~/TP3$

```

Figure 2: Hachage d'un même mot avec md5sum, sha1sum et sha256sum

On voit que les trois algorithmes donnent des résultats de longueurs différentes. L'algorithme recommandé par le NIST est **SHA-256** – MD5 et SHA-1 ne sont plus considérés comme sûrs aujourd'hui.

2.3 Génération d'un dictionnaire et attaque par dictionnaire

Pour l'attaque par dictionnaire, j'ai utilisé CeWL pour générer un dictionnaire à partir du site efrei.fr. CeWL scrape le site et récupère les mots qu'il trouve. Avec `-d 2` il descend sur 2 niveaux de pages, et `-m 4` ne garde que les mots d'au moins 4 caractères.

```
cewl -d 2 -m 4 https://www.Efrei.fr -w mon_dictionnaire.txt
```

```

Traitement des actions différées (« triggers ») pour fontconfig (2:17.1-subfont) ...
maziengubuntu-VM:~/TP3$ cewl -d 2 -m 4 efrei.fr -w ~/TP3/dictionnaire.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@dig.ninja) (https://dig.ninja/)
maziengubuntu-VM:~/TP3$ cat ~/TP3/dictionnaire.txt | head -20
maziengubuntu-VM:~/TP3$ wc -l ~/TP3/dictionnaire.txt
0 /home/mazien/TP3/dictionnaire.txt
maziengubuntu-VM:~/TP3$ cewl -w mon_dictionnaire.txt -d 2 -m 4 https://www.Efrei.fr
CeWL 6.2.1 (More Fixes) Robin Wood (robin@dig.ninja) (https://dig.ninja/)
maziengubuntu-VM:~/TP3$ cat ~/TP3/dictionnaire.txt | head -20
maziengubuntu-VM:~/TP3$ ls
01_hachage_base.py          05_hachage_salt_bcrypt.py    dictionnaire.txt
02_encodage.py             'attaque par dictionnaire.py' keylogger.py
03_chiffrement_symetrique_aes.py bruteforce.py                mon_dictionnaire.txt
04_chiffrement_asymetrique_rsa.py bruteforce_dictionnaire.py
maziengubuntu-VM:~/TP3$ cat ~/TP3/mon_dictionnaire.txt | head -20
Efrei
Mastère
vous
Manager
Bachelor
Google
Bordeaux
campus
Management
pour
école
votre
Campus
Prog
Cybersécurité
Data
Numérique
Spécialisé
International
numérique
maziengubuntu-VM:~/TP3$

```

Figure 3: Dictionnaire généré par CeWL – mots extraits du site efrei.fr

Ensuite j'ai lancé l'attaque. Le principe est simple : le script hash chaque mot du

dictionnaire avec MD5, et compare ce résultat avec le hash du mot de passe qu'on a saisi. Quand ça correspond, il affiche le mot de passe trouvé.

```
python3 "attaque par dictionnaire.py"
```

A terminal window with a dark purple background. The prompt is 'marlen@ubuntu-VM:~/TP3\$'. The user enters 'python3 "attaque par dictionnaire.py"'. The output is: 'The hash of your Password : 3e787dd18e364fe1129dd1eb11622c18 was Cracked, your password : Mastère'. The prompt returns to 'marlen@ubuntu-VM:~/TP3\$' with a cursor.

```
marlen@ubuntu-VM:~/TP3$ python3 "attaque par dictionnaire.py"
The hash of your Password : 3e787dd18e364fe1129dd1eb11622c18
was Cracked,
your password : Mastère
marlen@ubuntu-VM:~/TP3$
```

Figure 4: Mot de passe "Mastère" retrouvé avec succès

Le mot de passe *Mastère* a bien été retrouvé. C'est un bon exemple pour comprendre pourquoi il ne faut pas choisir un mot de passe tiré du contexte dans lequel on travaille.

2.4 Attaque par brute force

L'attaque par brute force est différente : au lieu d'une liste de mots connus, elle essaie toutes les combinaisons possibles de caractères. C'est garanti de trouver à un moment, mais ça peut prendre très longtemps selon la longueur du mot de passe. J'ai mis `ab` pour que ça se termine rapidement.

```
python3 bruteforce.py
```

```

##### ['?', '7'] #####
##### ['h', '~'] #####
##### ['.', 'n'] #####
##### ['2', 'f'] #####
##### ['x', 'v'] #####
##### ['c', 'k'] #####
##### ['e', 'u'] #####
##### ['\\', 'j'] #####
##### ['C', 'n'] #####
##### ['*', '4'] #####
##### ['y', '\\xc'] #####
##### ['-', 'g'] #####
##### ['L', '0'] #####
##### ['a', '^'] #####
##### ['o', 'F'] #####
##### ['A', 'l'] #####
##### ['!', ''] #####
##### ['J', 'v'] #####
##### ['\\c', 'T'] #####
##### ['"', 'V'] #####
##### ['X', 's'] #####
##### ['C', 'e'] #####
##### ['=', 'r'] #####
##### [')', ''] #####
##### ['Q', 's'] #####
##### ['X', 'i'] #####
##### ['V', 'F'] #####
##### ['9', 'H'] #####
##### ['p', 'z'] #####
##### ['Q', 'S'] #####
##### ['8', 'n'] #####
##### ['.', '0'] #####
##### ['8', 'B'] #####
##### ['2', 'p'] #####
##### ['/', 'J'] #####
##### ['x', 'g'] #####
##### ['^', '4'] #####
##### ['n', '0'] #####
##### ['w', '~'] #####
##### ['+', '3'] #####
##### ['e', '~'] #####
##### ['J', 'l'] #####
##### ['0', ':'] #####
##### ['\\n', 'y'] #####
##### ['l', ''] #####
##### ['.', 'X'] #####
##### ['A', 'q'] #####
##### ['E', 'B'] #####
##### ['v', '\\xc'] #####
##### ['t', '\\'] #####
##### ['t', '7'] #####
##### ['H', 'H'] #####
##### ['"', 'P'] #####
##### ['1', 'R'] #####
##### ['6', 'l'] #####
##### ['H', '0'] #####
##### ['v', 'v'] #####
##### ['q', 'H'] #####
##### ['l', 'x'] #####
##### ['-', 'E'] #####
##### ['>', 't'] #####
##### ['Q', '0'] #####
##### ['a', 'b'] #####
Votre mot de passe est : ab
mazien@ubuntu-VM: ~/TP3

```

Figure 5: Brute force – toutes les combinaisons testées jusqu'à trouver "ab"

La différence principale avec l'attaque par dictionnaire : le dictionnaire est très rapide mais ne trouve que les mots connus, tandis que le brute force trouve toujours mais est beaucoup plus lent.

2.5 Protection avec le Salt et bcrypt

Pour se protéger contre les attaques par dictionnaire, on utilise un salt. C'est une chaîne aléatoire unique générée pour chaque utilisateur, qu'on ajoute au mot de passe avant de le hasher. Du coup même si deux personnes ont le même mot de passe, leurs hashes seront différents. Les rainbow tables (des dictionnaires de hashes pré-calculés) ne servent plus à rien dans ce cas.

bcrypt fait la même chose mais en plus il est volontairement lent, ce qui rend les attaques massives beaucoup plus coûteuses.

```
python3 05_hachage_salt_bcrypt.py
```

```

nazien@ubuntu-VH:~/TP3$ python3 05_hachage_salt_bcrypt.py
SA4 - Script 05 : Hachage avec Salt & Bcrypt

--- POURQUOI LE SALT ? ---
Même mot de passe 'password123' hashé deux fois :
Hash 1 (salt=1537be1e41d2d6ce...) : e0df024e27be7505e35e64fc60c61e70b3d0b666e4ba8b11da18f290c2cdbab2
Hash 2 (salt=2311f33850c4d6bd...) : db00eeceb80d1cf2edc65844cadebf7ec8b63855174481110269797c29fc6774
→ Les hashes sont DIFFÉRENTS grâce au salt !

--- VÉRIFICATION AVEC SALT ---
Bon ndp : ✓
Mauvais ndp: ✗

--- HACHAGE BCRYPT (recommandé en production) ---
Hash bcrypt : $2b$10$7g08xbHZVNPwd5XZsBJte0uU0pLM3Z1JRPujT7H8qRVf3leTYeFDu
Temps de calcul (rounds=10) : 93 ns → intentionnellement lent !
Vérification OK : ✓

--- SIMULATION BASE DE DONNÉES UTILISATEURS ---
[OK] Utilisateur 'alice' créé.
[OK] Utilisateur 'bob' créé.

Connexion alice / alice2025! → ✓ Succès
Connexion alice / mauvais → ✗ Échec

✓ Démonstration terminée. Fichier users_db.json créé.
nazien@ubuntu-VH:~/TP3$

```

Figure 6: Même mot de passe haché deux fois – deux résultats différents grâce au salt

2.6 Attaque de collision sur MD5

Sensibilité aux modifications

Avant la collision à proprement parler, on commence par montrer que le hash est très sensible aux modifications. J'ai créé deux fichiers avec un contenu presque identique – juste un espace en plus dans le deuxième.

```

echo "test" > test1.txt
echo "test " > test2.txt
cat test1.txt && cat test2.txt
diff test1.txt test2.txt
md5sum test1.txt test2.txt

```



```

nazien@ubuntu-VM:~/TP3$ echo "test" > test1.txt
echo "test " > test2.txt
cat test1.txt
cat test2.txt
test
test
nazien@ubuntu-VM:~/TP3$ diff test1.txt test2.txt
1c1
< test
+++
> test
nazien@ubuntu-VM:~/TP3$ md5sum test1.txt test2.txt
d8e8fca2dc0f896fd7cb4cb0031ba249  test1.txt
b74ed347398d35c29923e80f64c9d904  test2.txt
nazien@ubuntu-VM:~/TP3$

```

Figure 7: cat ne montre aucune différence visible entre les deux fichiers

```

nazien@ubuntu-VM:~/TP3$ echo "test" > test1.txt
echo "test " > test2.txt
cat test1.txt
cat test2.txt
test
test

```

Figure 8: diff détecte l'espace supplémentaire dans test2

```

nazien@ubuntu-VM:~/TP3$ md5sum test1.txt test2.txt
d8e8fca2dc0f896fd7cb4cb0031ba249  test1.txt
b74ed347398d35c29923e80f64c9d904  test2.txt
nazien@ubuntu-VM:~/TP3$ █

```

Figure 9: Un simple espace change complètement le hash MD5

Collision avec l'outil md5collgen

La vraie collision c'est autre chose : générer deux fichiers avec des contenus *différents* qui donnent *exactement le même* hash MD5. C'est possible avec l'outil fourni par Souheib.

```

cd collgen/collgen/md5collgen-master && make
./md5collgen file.txt
xxd file_msg1.txt | head && xxd file_msg2.txt | head
md5sum file_msg1.txt file_msg2.txt

```



```

mazien@ubuntu-VM:~/TP3$ xxd file_msg1.txt | head
00000000: 7465 7374 0a00 0000 0000 0000 0000 0000  test.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000040: d370 c460 dffb 5f0e f90b 7c88 11a0 a43a  .p.`.._...|....:
00000050: 275c 4e16 4e30 06fd 42ca 401a f50c 16f0  '\N.N0..B.@.....
00000060: f8c9 a70c ec0c 1efd 4f0e def9 9bad 361b  ....O.....6.
00000070: 8ece 8735 82fb 248b 4036 4879 cde4 9eb5  ...5..$.@6Hy....
00000080: baba c16c 94b4 5ee8 c63e d445 2ed9 1aac  ...l..^...>.E....
00000090: a91c d784 df67 9dcf 86d5 bb4b 5f5c 4541  ....g.....K_\EA
mazien@ubuntu-VM:~/TP3$ xxd file_msg2.txt | head
00000000: 7465 7374 0a00 0000 0000 0000 0000 0000  test.....
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ....
00000040: d370 c460 dffb 5f0e f90b 7c88 11a0 a43a  .p.`.._...|....:
00000050: 275c 4e96 4e30 06fd 42ca 401a f50c 16f0  '\N.N0..B.@.....
00000060: f8c9 a70c ec0c 1efd 4f0e def9 9b2d 371b  ....O.....-7.
00000070: 8ece 8735 82fb 248b 4036 48f9 cde4 9eb5  ...5..$.@6H....
00000080: baba c16c 94b4 5ee8 c63e d445 2ed9 1aac  ...l..^...>.E....
00000090: a91c d704 df67 9dcf 86d5 bb4b 5f5c 4541  ....g.....K_\EA
mazien@ubuntu-VM:~/TP3$ md5sum file_msg1.txt file_msg2.txt
f49d214fd175d42df6a6b7c45197520c  file_msg1.txt
f49d214fd175d42df6a6b7c45197520c  file_msg2.txt
mazien@ubuntu-VM:~/TP3$

```

Figure 10: Deux fichiers différents en hexadécimal, mais même hash MD5

On voit clairement dans le dump hexadécimal que les deux fichiers ne sont pas identiques, et pourtant `md5sum` retourne exactement le même hash pour les deux. C'est pour ça que MD5 ne doit plus être utilisé pour vérifier l'intégrité de fichiers.

3. Encodage

L'encodage c'est juste une façon de représenter des données dans un autre format. Ce n'est pas du chiffrement – n'importe qui peut décoder sans clé. Le script montre quatre exemples.

```
python3 02_encodage.py
```

```

SA4 - Script 02 : Encodage

=====
Texte original : 'Hello EFREI!'
=====

Base64 :
  Encodé : SGVsbG8gRUZSRUkh
  Décodé : Hello EFREI!

Hexadécimal :
  Encodé : 48656c6c6f20454652454921
  Décodé : Hello EFREI!

Base32 :
  Encodé : JBSWY3DPEBCUMUSFJEQQ====
  Décodé : Hello EFREI!

ROT-13 (décalage de 13) :
  Encodé : Uryyb RSRV!
  Décodé : Hello EFREI! (ROT-13 appliqué deux fois)

⚠ L'encodage N'EST PAS du chiffrement – il est réversible sans clé !

=====
Texte original : 'Bonjour les étudiants'
=====

Corbeille : Qm9uan91ciBsZXMgw6l0dMRpYw58cw==
  Décodé : Bonjour les étudiants

Hexadécimal :
  Encodé : 426f6e6a6f7572206c657320c3a974756469616e7473
  Décodé : Bonjour les étudiants

Base32 :
  Encodé : IJXW42TPOVZCA3DFOMQMHKLUOVSGSYLOORZQ====
  Décodé : Bonjour les étudiants

ROT-13 (décalage de 13) :
  Encodé : Obawbhe yrf éghqvnagf
  Décodé : Bonjour les étudiants (ROT-13 appliqué deux fois)

⚠ L'encodage N'EST PAS du chiffrement – il est réversible sans clé !

=====
Texte original : 'P@ssw0rd123'
=====

Base64 :
  Encodé : UEBzc3cwcnQxMjM=
  Décodé : P@ssw0rd123

Hexadécimal :
  Encodé : 5040737377307264313233
  Décodé : P@ssw0rd123

Base32 :
  Encodé : KBAHG43XGBZGIMJSGM=====
  Décodé : P@ssw0rd123

ROT-13 (décalage de 13) :
  Encodé : C@ffj0eq123
  Décodé : P@ssw0rd123 (ROT-13 appliqué deux fois)

⚠ L'encodage N'EST PAS du chiffrement – il est réversible sans clé !
mazien@ubuntu-VM: ~/TP3$

```

Figure 11: Encodage et décodage Base64, Hexadécimal, Base32 et ROT-13

- **Base64** : sert notamment pour les pièces jointes dans les emails
- **Hexadécimal** : représentation classique des données binaires
- **Base32** : utilisé dans les codes d'authentification à deux facteurs
- **ROT-13** : décalage de 13 lettres, pas d'utilité sécuritaire réelle

4. Chiffrement et Signature d'Email

4.1 Installation et configuration de Thunderbird

J'ai installé Thunderbird et configuré mon adresse Gmail avec le protocole IMAP. Ensuite dans les paramètres du compte, section *Chiffrement de bout en bout*, j'ai créé une nouvelle clé OpenPGP RSA 4096 bits.

La logique derrière : on a deux clés liées. La clé publique peut être partagée avec tout le monde – elle permet de chiffrer des messages à notre attention. La clé privée reste secrète et sert à déchiffrer. Personne d'autre ne peut lire le message même s'il est intercepté.

4.2 Envoi d'un email chiffré

J'ai envoyé un email à ma propre adresse avec le bouton *Chiffrer* activé dans la barre d'outils de Thunderbird.

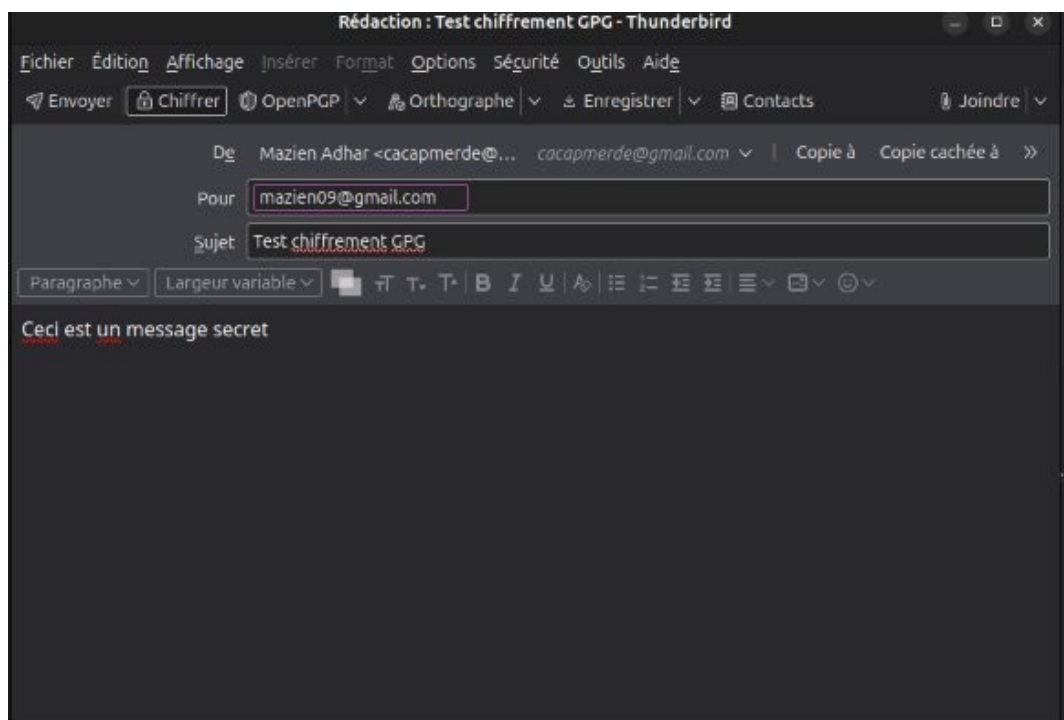


Figure 12: Email prêt à être envoyé avec le chiffrement activé

4.3 Consultation sur Gmail – contenu illisible

En ouvrant le même email depuis Gmail dans Firefox, le contenu est complètement illisible. C'est normal : Gmail n'a pas la clé privée, donc il ne peut pas déchiffrer. Même si quelqu'un accède au compte Gmail, les emails chiffrés restent inaccessibles.

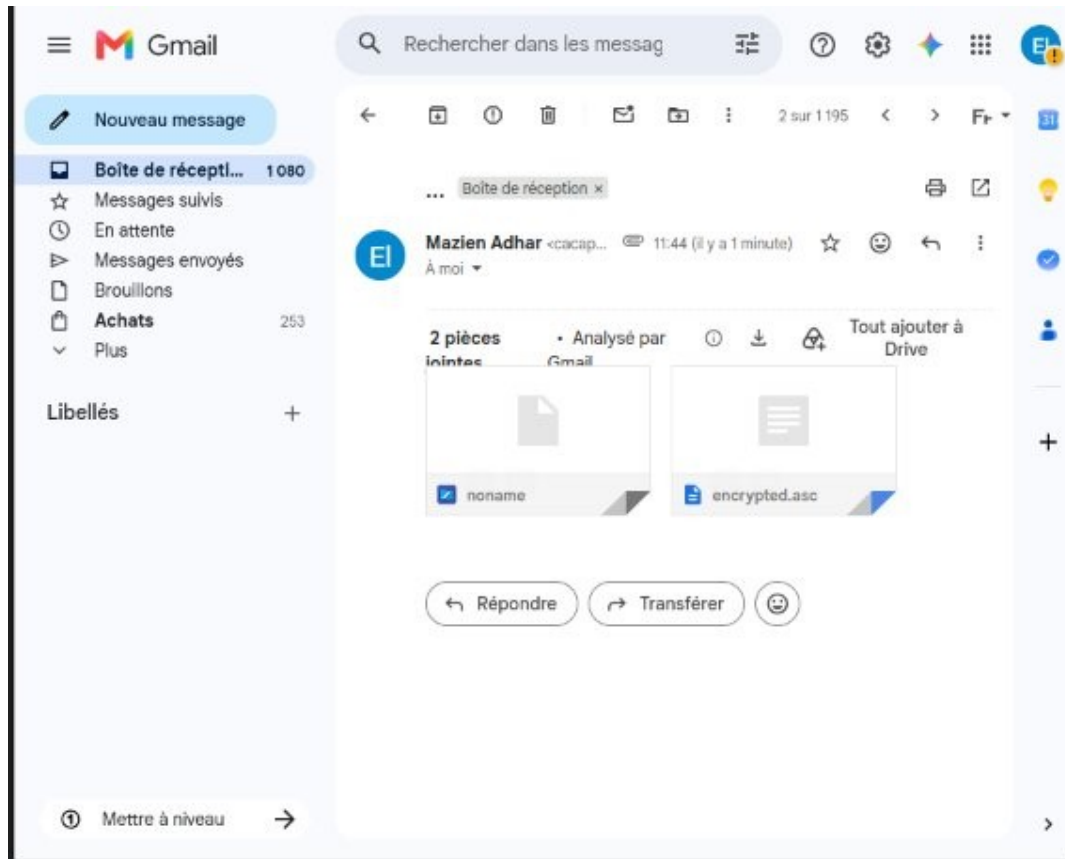


Figure 13: Le contenu PGP est illisible depuis Gmail

4.4 Déchiffrement en ligne de commande avec GPG

J'ai exporté la clé privée depuis le Gestionnaire de clés OpenPGP de Thunderbird, puis je l'ai importée dans GPG pour pouvoir déchiffrer le fichier directement en ligne de commande.

```
gpg --import ~/T1chargements/cle_privee.asc
gpg --decrypt ~/T1chargements/encrypted.asc
```



```

Message-ID: <b7ee9df1-cc83-423b-acad-cf3c6b9223b8@gmail.com>

-----lvBN6b6hh106ez6V0jWsw0I
Content-Type: multipart/mixed; boundary="-----zraJDUpRn29iYdMw0sZ4hSoG"

-----zraJDUpRn29iYdMw0sZ4hSoG
Content-Type: text/plain; charset=UTF-8; format=flowed
Content-Transfer-Encoding: 7bit

Ceci est un message secret

-----zraJDUpRn29iYdMw0sZ4hSoG
Content-Type: application/pgp-keys; name="OpenPGP_0x525483887B2DEB90.asc"
Content-Disposition: attachment; filename="OpenPGP_0x525483887B2DEB90.asc"
Content-Description: OpenPGP public key
Content-Transfer-Encoding: 7bit

-----BEGIN PGP PUBLIC KEY BLOCK-----

xsDNBComjjcBDACs6TMY0cbfA6HfLK46G+/fId6D7JEHPsp10D4jGpt0SY+97/5A
gtRz4wmir511UTapCLISicyUiHNUHwEqAf1y3GFPeTPlYVYkF0sxZTM0rf3QYmoF
S0pctNy0vp21LHT70p2PkMSEPXzr3PBAyM2JwrjQU6Ry/ow9cp+AAHLUhx176ePu
x0BRacLyfFR00uHyrbMI5wRpXq1Hox+2YJd7rTI9pE1fzDTbhrvNfLhPIYJRIDJ
GAzs2A538nVC0nkrca1a6R6evssXAhw3712zJe7GkbDn1t+eAVMVXNoYkGp7IRfni
/cLWQ1H0cUE74nUNTCgHZjB9mPu5KrI814A+thfDA6BPxDawSL51XIdOPzZ+VrD
+1doFCRqKfC0T7VCTBoGinuJD3bYfSYbA0bxeqmeJIgnE9+Rnt/XbNzPEtU9/5l+
7hcc4EKcPmZwusiEJYQQ202saxT6ddl/cqPzYtZTcHLARt8FzQAZYXMMvR5Ay+e8
j2YCjJm5nsF5+80AEQEAAC0jTWF6awVuIEFkaGFyIDxjYWNhcG1lcmRlQGdtYVls
LnNvbT7CwQ0EFwEIA0cWlQTTDGA9HwTmPw7cQbdSVI07ey3rkAUCaia0NwUJ8a0a
gAIBAwQLCQgHBRUICQoLBRVCAwEAAoJEFJUG7t7LeuQafAMAKmHTUeENlz51srv
ocEWSg612CaTbF2pZrLdbKiJqQ0kmMnpZAKE+c5zqExB+vNtvCqHFYH7Ze4rY32I
xqJ/wtQkZHTJwHehWwMvCkM8Q128uB31Dn9s4ygrCkuuw2gES2L34ymzj+IX26
cBHacFc36XQq0BZ0mrBRv/fHauh2vP4vrIP1vk0KcIwCIjYmRyZp6tS06/+dJwb
z460vm3K+gb1FCLIPwkbhFwngL67cN085EDvZvXxueSWa9oqwg68MydW9UtxNeI
/QLl90fxdye0BR+445sm7LAIImS87uh+K6bnY9TlRr/UA0xUzq3GdlL7+28kep
6KvE9K255j2bGDoLY0SuoSHsji1+awCR8guTPw18mXMS7qKpGJqs0XhgFhoYac2k
FSRkERXmWuW0DzLdieLP2L/eHmzk+Suwy8n89mWyEdcc9icA54a9+PakCxpAUjPU
Q0sDj5l000VJ1GJ0+CSKVhBDG+qh7S8FcrnoUHPmqnwFES4P7s7AZQRqJo44AQWA
ur+dkzv2nexGyZkklIYijjRGR+8ycmvPyMSTVMi86faeFJd9SG00AT3I3Enw8+Q
6r2iBIRdRFOALdkTJl4HRkLJMvO3WqFgBBYJw70FcguT1NnwKBx52HE0tpK+lsqx
jhYURgtfn8+Keoz7YKehP8qVhsz8YvAl1l2tejjURWycNcxf0wYkwJNt3vBAGZE
L5scrgfFeX58EPci330SU7clC7Ip4JmsD3tR4ITR+9RCH6/cYVWYCGkhGgYcxEHn
r4LW1tdJWWEFjRw1GymyKfgJCCGauZfc3M2FzuXU1v0tW0vgbRW3Wv2lddaxkf
16/a1vqloKSNaPe1838zy2m/LwprD015tM8u+2V4F7hrc260mm4yuT6b3cX9CxrM
lBog5r8hxb7NzG3x48BqQ4BFLTmvuRvkSqJnTQ4EqL6GRmEyYJad30tyb7FJ5aFZ
+1wmNvvpBPVbhnH0pceFuYSR2A0EoRP4BCLPkHJxsaVHgPYjdcwBv2GHV05tn
ABEBAACwPwEGAEIACYWIQTDDGa9HwTmPw7cQbdSVI07ey3rkAUCaia00QUJ8a0a
gAIBDAACRBSVIO7ey3rkFmfDACTNd/Ofvtl21n1VTtZVvPhJADdk0Xuw2xwZfAS
mz0dufKUXvTLGJAQ/c4yR2Q7d8PMLf4v3qYu1qW8mgcFNEckRc0oDULTwfWz259
ck6GDcThaNePE0zQ0/Lb0tzuRKJM4S71qSYR07MKXfCuzH2MfhfQ3ZZ3p9G1voFk
fLhT4NK3/bHaY29rrQnwtL9b+SqXu/fwe5HPQwB1TP4/pFE66Iw4S0885Cjxm69
JKEtsHfuLB8AZtc+5xIp6yHBFXzuDRZVcz5rJ0EBDLpErRmHeub9dq4eJy5ZCDXH
HhGh1V8b0o50vsnMuxcdlQf6LaqnrSCU6ryGrwa0cAla+h/dSjYbwhyFrh08E0s
76gPBvZ4QmIE+0ashMcC/W5npVpECqkzavQlWovZaXluunD2WqCOlKhmlH9IsSdu
t6CvPRsP3dNRtCB6x/YxHkeGP3QuBibu0oVjho0KIjxgVNPiCvDeKygn6NvVX9G
B8egQ0WMP0srJeaN6xASJUpy1Po=
=Kaal
-----END PGP PUBLIC KEY BLOCK-----

-----zraJDUpRn29iYdMw0sZ4hSoG--

-----lvBN6b6hh106ez6V0jWsw0I--
gpg: Signature faite le lun. 08 juin 2026 11:44:31 CEST
gpg: avec la clef RSA D30C66B41F04E63D6EDC41B7525483887B2DEB90
gpg: Bonne signature de « Mazien Adhar <cacapmerde@gmail.com> » [inconnu]
gpg: Attention : cette clef n'est pas certifiée avec une signature de confiance.
gpg: Rien n'indique que la signature appartient à son propriétaire.
Empreinte de clef principale : D30C 66B4 1F04 E63D 6EDC 41B7 5254 8388 7B2D E890
mazien@ubuntu-VM:~/TP3$

```

Figure 14: Message déchiffré en ligne de commande – "Ceci est un message secret"

Le message "*Ceci est un message secret*" s'affiche bien en clair. La signature GPG est aussi validée, ce qui confirme que le message vient bien de l'expéditeur attendu.

5. Conclusion

Ce TP m'a permis de voir concrètement comment fonctionnent les trois mécanismes de base de la sécurité des données.

Pour le hachage, le plus important à retenir c'est que SHA-256 est l'algorithme recommandé aujourd'hui, et que MD5 ne doit vraiment plus être utilisé pour des usages sécuritaires vu qu'on peut générer des collisions. Le salt et bcrypt sont des protections supplémentaires utiles surtout pour les mots de passe stockés en base de données.

L'encodage ne protège rien, il faut juste ne pas le confondre avec du chiffrement.

Enfin la partie Thunderbird/GPG est celle qui m'a le plus parlé parce qu'elle montre un cas d'usage réel. Le fait de voir que Gmail lui-même ne peut pas lire un email chiffré illustre bien ce que "chiffrement de bout en bout" veut dire.